

INDEPENDENT UNIVERSITY

An Undergraduate Internship Project on Orient Computer E-commerce Website

By

Sultana Tasnim Riza

Student ID: 2111329

Summer 2026

Supervisor:

Bijoy Rahman Arif

Senior Lecturer

Department of Computer Science & Engineering

Independent University, Bangladesh

12 September, 2026

*Dissertation submitted in partial fulfillment for the degree of Bachelor of Science in
Computer Science*

Department of Computer Science & Engineering

Independent University, Bangladesh

Attestation

I, **Sultana Tasnim Riza**, ID: **2111329**, hereby declare that the report titled “**Orient Computer E-commerce Website**” submitted in partial fulfillment of the requirement for the Degree of Bachelor of Science in Computer Science is the result of my own work. I also confirm that the work was carried out as a student of Independent University, Bangladesh.

I would like to acknowledge the guidance of my supervisor, Bijoy Rahman Arif, who supported me throughout this project. I also declare that this work represents my personal understanding and the outcomes I achieved during my internship period.

The report also properly cites and credits all sources used for further inquiry about this report, contacting my supervisor in the internship setting, Md Sabbir Hasan Khan at Orient Computers through official communication channels.

Signature

Date

Sultana Tasnim Riza
Name

Acknowledgement

I am grateful for the opportunity to complete this internship and produce the project described in this report. My thanks go first to the Almighty and to my family, who were patient and encouraging throughout the long hours of software development, debugging, and report writing.

My academic supervisor, **Bijoy Rahman Arif**, provided clear direction and asked the right questions at each review milestone. Without those check-ins the project would have drifted in several unhelpful directions.

I also thank the team at **Orient Computers**, particularly **Md Sabbir Hasan Khan**, for explaining the business requirements honestly. He helped me understand how orders are fulfilled, how district-wise delivery charges work in Bangladesh, and how manual bKash and Nagad payment verification is handled. His guidance encouraged me to work more carefully and confidently.

I express my sincere gratitude to the faculty members of the Department of Computer Science & Engineering at Independent University, Bangladesh for establishing strong foundational concepts in web application design, data structures, and database management systems.

Letter of Transmittal

12 September, 2026

Bijoy Rahman Arif

Senior Lecturer

Department of Computer Science & Engineering, School of Engineering and Computer Science

Independent University, Bangladesh

Subject: Submission of an internship report for graduation completion

Dear Sir,

I am submitting my Internship Report as part of the Bachelor Program in Computer Science. I am grateful for your supervision during my internship at Orient Computers, where I worked for three months under the guidance of Md Sabbir Hasan Khan.

The project involved designing and building a full-stack e-commerce web application for Orient Computer, a computer hardware and electronics retailer based in Dhaka, Bangladesh. The application lets customers browse products, place orders, and pay using Cash on Delivery, bKash, or Nagad. An administrative dashboard allows staff to manage the full order lifecycle including payment verification, product cataloging, stock control, and localized shipping fees.

I have tried to document the work clearly, including the decisions I made, the problems I faced, and how I solved them. If any section needs clarification, I would be happy to discuss it.

Thank you for your valuable guidance and consideration.

Yours sincerely,

Sultana Tasnim Riza

ID: 2111329

Department of Computer Science & Engineering,
Independent University, Bangladesh

Evaluation Committee

Supervision Panel

..... Academic Supervisor	 Industry Supervisor
------------------------------	------------------------------

Panel Members

..... Panel Member 1	 Panel Member 2
..... Panel Member 3	 Panel Member 4

Office Use

..... Internship Coordinator	 Head of the Department
---------------------------------	---------------------------------

Abstract

This report describes my work on designing and developing a full-stack e-commerce website for Orient Computer, a technology retailer in Dhaka, Bangladesh. The company sells PC components, UPS/IPS power backup systems, industrial batteries, solar equipment, and office technology. Before this project, most sales activity depended on showroom visits, phone inquiries, and manual order handling, so there was a clear need for a more organized online sales and order management system.

The system was built using a three-tier MERN stack architecture: MongoDB Atlas for the database, Express.js and Node.js for the backend API, and React 18 with Vite for the frontend. Tailwind CSS was used for styling. The customer side of the application supports product browsing, category and brand filtering, search, cart management, and checkout. Since the project was built for the Bangladeshi market, the checkout process includes Cash on Delivery (COD), bKash, and Nagad payment options with transaction proof submission.

An admin dashboard was also developed so staff can manage categories, products, stock, delivery charges, coupons, customer reviews, and order status updates. Security was handled through bcrypt password hashing, JSON Web Token (JWT) authentication, role-based access control, input sanitization, and server-side validation of prices and stock. Testing was carried out on the main customer and admin workflows to check that the system worked reliably across the most important use cases.

Keywords: MERN Stack, E-commerce, Bangladesh, bKash, Nagad, React, Node.js, Express.js, MongoDB Atlas, JWT Authentication, Role-Based Access Control.

Contents

Attestation	i
Acknowledgement	ii
Letter of Transmittal	iii
Evaluation Committee	iv
Abstract	v
1 Introduction	1
1.1 Overview and Background	1
1.2 Objectives	1
1.3 Scope	2
2 Literature Review	3
2.1 Relationship with Undergraduate Studies	3
2.2 E-Commerce Context in Bangladesh	3
2.3 Related Works and Technical Landscape	4
2.4 Existing Website Analysis	4
2.5 Competitive Analysis	6
3 Project Management and Financing	7
3.1 Work Breakdown Structure	7
3.2 Process and Activity Time Distribution	7
3.3 Gantt Chart	8
3.4 Resource Allocation and Estimated Costing	8
4 Methodology	9
4.1 Architecture Overview	9
4.2 Technology Stack	9
4.3 Database Design and API Structure	10
5 Body of the Project	11
5.1 Work Description	11
5.2 Requirement Analysis	11
5.2.1 Functional Requirements	11
5.2.2 Non-Functional Requirements	12
5.3 System Analysis and Feasibility	12
5.4 System Design and Data Modeling	12
5.4.1 MongoDB Collection Schema Summary	13
5.5 UI Page Mockups	13
5.6 Order State Machine	17

5.7	Key Code Implementations	18
5.7.1	User Schema with bcrypt Password Hashing	18
5.7.2	JWT Authentication Middleware	19
5.7.3	Product Schema with Virtuals and Text Index	20
5.7.4	Order Creation with Guest Checkout and Atomic Stock Decrement	21
5.7.5	Admin Analytics Aggregation Pipeline	22
5.8	Implementation and Testing	23
6	Results and Analysis	25
6.1	System Features Delivered	25
6.2	Deployment Architecture	26
7	Project as Engineering Problem Analysis	27
7.1	Sustainability of the Project	27
7.2	Social and Environmental Considerations	27
7.3	Ethical and Legal Considerations	27
8	Lessons Learned	28
8.1	Problems Faced	28
8.2	Solutions and Retrospective Insights	28
9	Future Work and Conclusion	29
9.1	Future Work	29
9.2	Conclusion	29
	Bibliography	30

List of Figures

2.1	Conceptual Three-Tier Application Architecture	5
3.1	Work Breakdown Structure (WBS) Diagram	7
3.2	Project Gantt Chart	8
4.1	Detailed Component Architecture and Request/Response Flow	9
5.1	Rich Picture of Orient Computer E-commerce Platform	11
5.2	Entity Relationship Diagram (ERD)	12
5.11	Order Lifecycle Finite-State Machine	17
6.1	Production Deployment Architecture	26

List of Tables

2.1	Summary of Existing Orient Website Review	5
3.1	Time Distribution by Project Phase	7
3.2	Estimated Project Development and Hosting Cost	8
4.1	Technology Stack	10
4.2	Selected Core REST API Endpoints	10
5.1	MongoDB Collection Schema Summary	13
5.2	Permitted Order Status State Transitions	18
5.3	Key Test Case Summary	24
6.1	System Performance and Latency Metrics	25

Chapter 1

Introduction

1.1 Overview and Background

This internship gave me the opportunity to connect classroom learning with a real software project. As part of the graduation requirements for the Bachelor of Science in Computer Science at Independent University, Bangladesh, I completed a three-month internship at Orient Computers.

Orient Computers is a technology and electronics provider based in Dhaka, Bangladesh. The company's official website lists product categories including PC components, renewable energy products, UPS, IPS, batteries, telecom equipment, audio-visual products, and office equipment [2]. Although the business already had a strong product range, much of the sales process still depended on physical retail interactions and manual phone consultations.

Moving this type of retail operation online is not only a matter of creating product pages. In Bangladesh, many customers still prefer Cash on Delivery or Mobile Financial Services (MFS) such as bKash and Nagad instead of international card payments. Delivery charges also differ between Dhaka and other districts. For that reason, I designed the project around the actual business process rather than copying a generic e-commerce template.

1.2 Objectives

The primary objectives of the internship project were to:

- 1. Design and Develop a Responsive Customer Storefront:** Create a clear interface where customers can search, filter, view technical specifications, and manage carts on both desktop and mobile devices.
- 2. Implement Localized Payment & Checkout Processing:** Build a checkout flow that calculates delivery fees by district and supports COD, bKash, and Nagad payments.
- 3. Create an Administrative Operations Dashboard:** Give administrators tools for inventory tracking, category management, customer oversight, coupon management, and order status updates.
- 4. Build a Payment Verification Workflow:** Add admin review features for manual MFS transactions, including screenshot checking and payment state tracking.

- 5. Maintain Security and Data Integrity:** Use bcrypt password hashing, JWT authentication, role-based route guards, and server-side price validation to reduce the risk of client-side tampering.

1.3 Scope

The scope of the project includes:

- User authentication, profile management, and persistent shipping addresses.
- Product catalog with full-text indexing, multi-level categorization, brand filtering, and real-time stock indicators.
- Guest cart management in browser storage with automated server-side synchronization upon login.
- Order creation with atomic stock decrement and server-side price verification.
- Anonymous order tracking by order tracking number and phone number.
- Customer product reviews and wishlist management.
- Admin panel with analytics, low-stock warnings, and transaction moderation.

Out-of-scope items include third-party automated merchant aggregator APIs (e.g., SSLCommerz), automated SMS gateways, and interactive PC builder compatibility tools, which are reserved for subsequent release phases.

Chapter 2

Literature Review

2.1 Relationship with Undergraduate Studies

The Orient Computer e-commerce platform was a practical way for me to use many of the topics I studied during my undergraduate program. The project required requirements analysis, interface design, database modeling, API development, authentication, and basic security controls. These areas connect closely with several courses from the Bachelor of Science in Computer Science program.

- **CSE309 (Web Application Development):** This course helped me understand client-server communication, asynchronous JavaScript, RESTful API design, and component-based frontend development. These ideas were used directly while building the React frontend and Express backend [9, 10].
- **CSE307 (System Analysis and Design):** Requirements gathering, use-case modeling, rich pictures, and software requirements specification (SRS) work were useful before writing code. This helped me understand both the customer shopping flow and the internal admin workflow.
- **CSE303 (Database Management):** Even though this project uses MongoDB instead of a relational database, the database design concepts from the course were still important. Entity relationships, indexing, and query design helped me decide where to embed data and where to use references. Mongoose was used to add schema validation at the application level [11].
- **CSE211 (Object-Oriented Programming):** Modular programming ideas helped me separate backend models, controllers, middleware, and utility functions. This made the project easier to understand and maintain.
- **CSE403 (Information Security):** This course helped me think about password hashing, JWT authentication, role-based access control, server-side validation, and rate limiting. These decisions also match OWASP guidance on common web application risks [8].

2.2 E-Commerce Context in Bangladesh

E-commerce in Bangladesh has grown with wider internet access, smartphone use, and greater awareness of digital payment services. Government digital transformation initiatives have encouraged online service delivery and digital transactions, while private retailers have responded by building web-based sales channels [1]. For a computer and electronics retailer such as Orient Computer, an online platform needs more

than a simple product list. Customers expect searchable catalogues, accurate technical specifications, visible stock information, delivery options, and payment methods that match local habits.

Payment behavior is especially important in the Bangladeshi market. Cash on Delivery is still familiar to many customers, while Mobile Financial Services (MFS) such as bKash and Nagad are widely used for person-to-person and merchant payments. Bangladesh Bank recognizes MFS as part of the country's payment infrastructure, and local retailers often combine mobile payments with branch, delivery, or order-verification processes [3]. This is why the project includes COD, bKash, and Nagad instead of depending only on card-based checkout.

2.3 Related Works and Technical Landscape

Existing Bangladeshi technology retailers provide useful reference points for this project. Star Tech presents itself as a major computer, laptop, and gadget e-commerce and retail shop in Bangladesh, with online shopping support for desktops, laptops, office equipment, components, accessories, and related products [4]. Ryans Computers similarly operates an e-commerce platform and publishes payment method information that includes cash, bKash, Nagad, Rocket, and card options [5]. These competitors show that successful technology retail platforms in Bangladesh generally need deep category navigation, specification-heavy product pages, search and filtering, and locally appropriate payment support.

The Orient Computer platform adapts these market-validated patterns to the company's own product mix. Unlike general consumer marketplaces, the system must represent UPS, IPS, batteries, solar products, computer hardware, networking devices, and accessories. Such products often require technical attributes, model numbers, warranty details, and stock status. Therefore, the catalogue design emphasizes structured product specifications, category and brand filters, full-text search, and administrative inventory control.

2.4 Existing Website Analysis

An analysis of the official Orient Computers & Engineering website helped identify strengths that should be preserved and gaps that the internship project could address [2]. The existing website already presents Orient as a wholesale and retail technology store in Bangladesh and shows a wide product range across PC components, AVR, renewable energy, UPS, IPS, batteries, telecom products, audio-visual products, and office equipment. Its category structure is strong because the main navigation separates technical product groups clearly and uses subcategories such as online/offline UPS, solar inverters, solar panels, energy storage systems, and different battery types.

The website also shows several useful e-commerce and marketing practices. Product prices are displayed in Bangladeshi Taka, while high-value or business-oriented products may use "Call For Price," which makes sense for items that require quotation or consultation. The homepage includes featured categories, new-arrival and upcoming-product sections, live search with product previews, brand partner visibility, a phone number, a physical address, and social media links. These elements

support product discovery and customer trust, so I treated them as useful reference points while designing the new platform.

The review also showed some areas that could be improved. Some technical SEO elements appeared incomplete or inconsistent, such as empty metadata fields, weak social card configuration, and structured-data issues. From a user-experience perspective, the site would benefit from stronger product rating visibility, more consistent add-to-cart actions, completed company information, and clearer support pages such as FAQ, return policy, terms, and refund policy. These observations helped shape the project scope by showing that the system needed good product display, reliable search, mobile-friendly navigation, and better support for trust-building content.

Table 2.1: Summary of Existing Orient Website Review

Area	Observation
Navigation	Clear mega-menu structure with major product categories and relevant subcategories for technical retail browsing.
Product presentation	Pricing is visible in Bangladeshi Taka, and “Call For Price” is used for premium or quotation-based products.
Search and discovery	Live search, featured categories, product sections, and brand visibility help users discover products quickly.
Trust signals	Phone number, address, brand partners, and Facebook presence support credibility for local customers.
Improvement needs	SEO metadata, structured data, product reviews, policy pages, social links, and consistent product actions require further refinement.

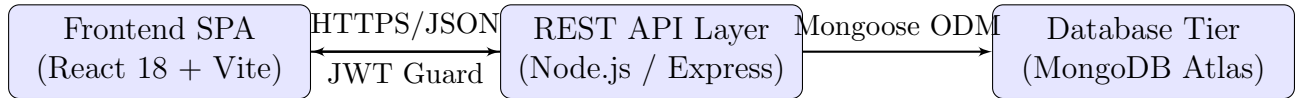


Figure 2.1: Conceptual Three-Tier Application Architecture

Figure 2.1 shows the three-tier architecture used in the project: a React single-page application, an Express/Node.js API layer, and a MongoDB database tier. I selected the MERN stack because it keeps both frontend and backend development in JavaScript, which was practical for a small internship project [6]. React supports reusable UI components for product cards, filters, forms, dashboards, and checkout screens [10]. Express provides simple routing and middleware patterns for authentication, validation, and REST API endpoints [9]. MongoDB supports flexible document structures, which is useful because product specifications vary across categories [11].

Authentication and authorization were also important. JSON Web Tokens (JWTs) are commonly used for stateless authentication in REST APIs because the server can verify signed claims without keeping session state for every request [7]. In this project, JWTs are combined with role-based middleware so customer routes and admin routes stay separate. Passwords are protected using bcrypt-style hashing, which is designed to be computationally expensive and adaptable as hardware improves [12]. I also kept price calculation, stock decrement, and order status transitions on the server so they could not be changed from the browser.

2.5 Competitive Analysis

The competitive review suggests that Orient Computer should not try to compete only through visual design. Larger retailers already have product range, brand recognition, and online ordering. For this project, the more realistic goal was to build a maintainable system that fits Orient Computer's daily workflow. The platform includes customer-facing features such as search, cart persistence, checkout, order tracking, wishlist, and reviews, while also supporting staff-facing features such as inventory monitoring, order moderation, coupon control, dashboard analytics, and manual MFS payment verification.

This focus is important because Orient Computer's problem is not limited to publishing products online. The business also needs to turn browsing into verified orders, handle district-based delivery costs, manage stock changes, and review customer-submitted mobile payment evidence. The literature and market review therefore support three design priorities for the project: a familiar e-commerce interface for customers, a secure API and database foundation, and an admin panel that matches the real workflow of a Bangladeshi electronics retailer.

Chapter 3

Project Management and Financing

3.1 Work Breakdown Structure

I organized the project into six main phases. Figure 3.1 shows how the work was divided.

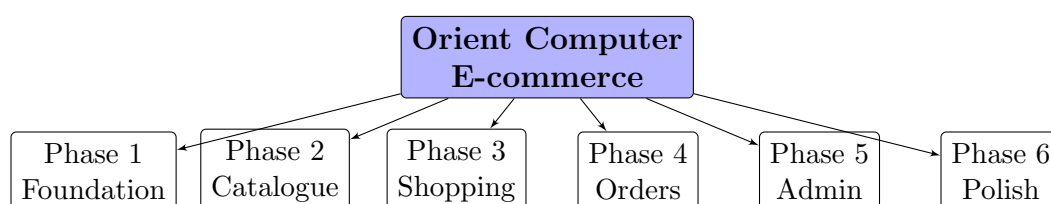


Figure 3.1: Work Breakdown Structure (WBS) Diagram

3.2 Process and Activity Time Distribution

Table 3.1 outlines the estimated versus actual timeline allocated to each project phase over the 13-week development period.

Table 3.1: Time Distribution by Project Phase

Phase Description	Estimated (Days)	Actual (Days)
Phase 1: Foundation (Setup, Auth, Base UI)	10	12
Phase 2: Catalogue (Categories, Products, Search)	14	16
Phase 3: Shopping (Cart, Checkout, Payments)	12	15
Phase 4: Orders (Creation, Tracking, Lifecycle)	10	11
Phase 5: Admin Panel (Management Dashboard)	16	18
Phase 6: Extras and Final Polish	8	10
Total Project Duration	70	82

3.3 Gantt Chart

Figure 3.2 shows the project timeline and the overlap between different phases.

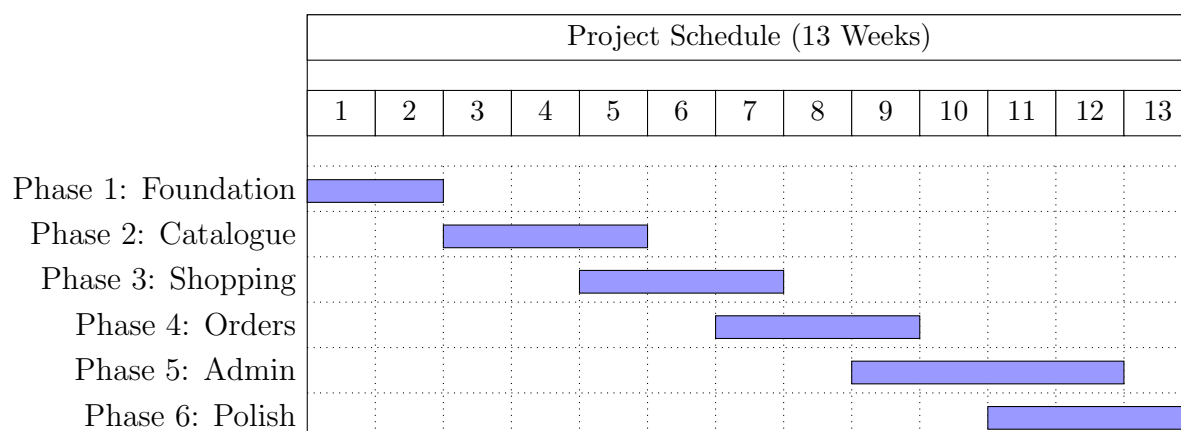


Figure 3.2: Project Gantt Chart

3.4 Resource Allocation and Estimated Costing

Hardware and infrastructure resources were carefully managed to minimize overhead during development.

Table 3.2: Estimated Project Development and Hosting Cost

Item Description	Estimated Cost (BDT)
Development Workstation (Existing Core i7, 16GB RAM)	0
Database Hosting (MongoDB Atlas M0 Free Cluster)	0
Development Software & Tooling (VS Code, Git, Postman)	0
Domain Registration (Annual .com/.com.bd)	~ 1,500
Frontend Hosting (Vercel Production Edge CDN)	0
Backend API Hosting (Railway/Render Container Instance)	~ 1,200 / month
Internet and Connectivity (3 Months Development)	~ 3,000
Total Setup Cost (Excluding Recurring Cloud Hosting)	~ 4,500 BDT

Chapter 4

Methodology

4.1 Architecture Overview

The application uses a three-tier client-server architecture. The browser communicates with the Node.js/Express backend through REST API endpoints. The backend handles validation and then communicates with MongoDB Atlas through Mongoose schemas. Figure 4.1 shows how the main components communicate with each other.

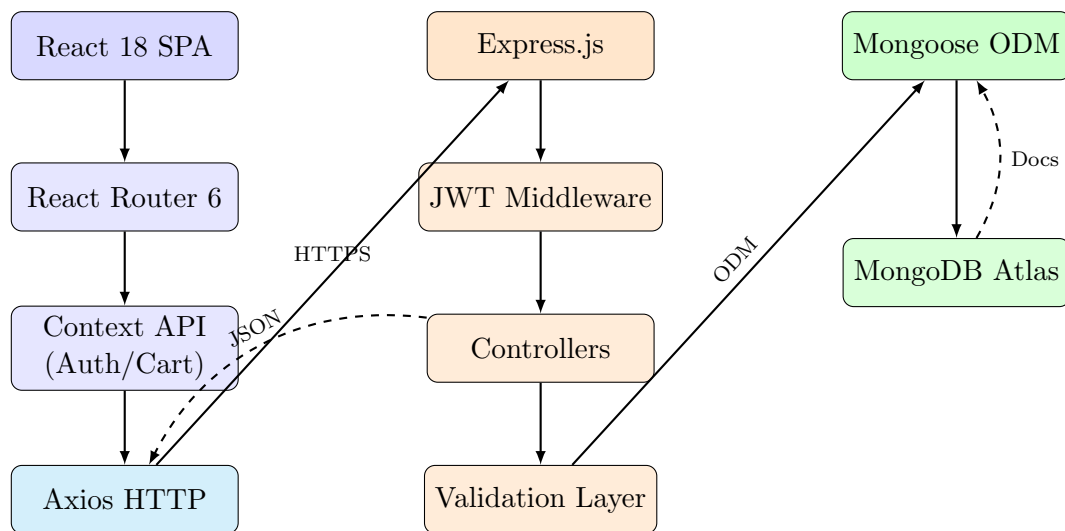


Figure 4.1: Detailed Component Architecture and Request/Response Flow

4.2 Technology Stack

Table 4.1 lists the main libraries and tools used in the project.

Table 4.1: Technology Stack

Layer	Technology	Rationale & Purpose
Frontend	React 18 with Vite	Component-based UI development and fast local development builds.
Routing	React Router 6	Declarative client-side routing with nested layout support.
Styling	Tailwind CSS 3	Utility-first styling for responsive layouts.
State & API	Context API + Axios	Shared auth/cart state and reusable HTTP request handling.
Backend	Node.js + Express.js	API routing, middleware, and backend business logic.
Database	MongoDB Atlas	Flexible schema support for different electronic product specifications.
Security	JWT + bcryptjs	Stateless authorization tokens and salted password hashing.
Sanitization	Helmet, Sanitize, Mongo-	HTTP security headers and NoSQL query injection prevention.

4.3 Database Design and API Structure

Orders store point-in-time snapshots of purchased product data, such as title, unit price, and SKU. This means old orders remain accurate even if a product title, price, or catalogue entry is changed later.

Table 4.2: Selected Core REST API Endpoints

Method	Endpoint Path	Description / Operation
POST	/api/auth/register	Register new customer account
POST	/api/auth/login	Authenticate credentials and return signed JWT
GET	/api/products	Query catalog with filtering, sorting, and pagination
GET	/api/products/:id	Retrieve single product details with reviews
POST	/api/orders	Create verified order with stock validation
POST	/api/orders/track	Public anonymous order tracking by number & phone
GET	/api/admin/dashboard	Fetch aggregate operational KPIs and recent orders
PATCH	/api/admin/orders/:id/status	Advance order lifecycle state with timeline logging
PATCH	/api/admin/orders/:id/payment	Approve or reject manual MFS transaction

Chapter 5

Body of the Project

5.1 Work Description

The main implementation work covered frontend components, Express controller logic, database schema design, and deployment preparation. One of the most important parts was the checkout flow, because it needed to create orders, reduce stock correctly, and keep the order status consistent.

5.2 Requirement Analysis

Figure 5.1 shows the system rich picture, including the main users, admin staff, and system components.

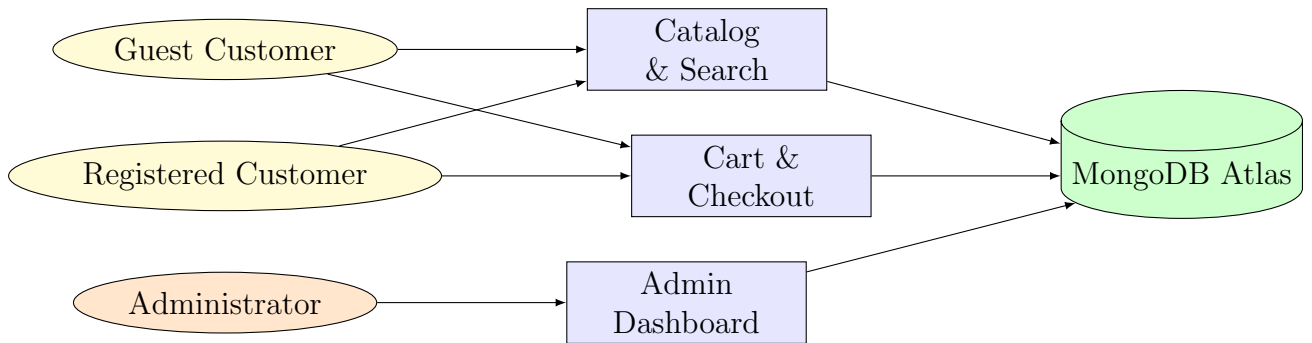


Figure 5.1: Rich Picture of Orient Computer E-commerce Platform

5.2.1 Functional Requirements

The application satisfies the following core functional requirements:

1. Customer account registration, login, and JWT-authenticated session persistence.
2. Hierarchical catalog browsing with dynamic filtering by category, brand, and price.
3. Technical specification displays tailored to diverse equipment categories.
4. Client-side guest cart storage with automatic synchronization upon account login.
5. Server-enforced checkout calculations with district-specific delivery charges.

6. Support for COD and manual MFS payments (bKash/Nagad) with screenshot upload.
7. Secure order tracking accessible via order reference and contact number.
8. Full administrative control over inventory, catalog taxonomy, coupons, and orders.
9. Controlled order status machine (Pending → Confirmed → Processing → Shipped → Delivered).

5.2.2 Non-Functional Requirements

1. **Responsiveness:** Fully adaptive layouts across mobile, tablet, and desktop screens.
2. **Localization:** Currency representations in Bangladeshi Taka (Tk.) and validation of Bangladeshi mobile phone formats (01XXXXXXXX).
3. **Performance:** Page load completion within 3 seconds over 4G connections.
4. **Security:** Passwords hashed with bcrypt (cost factor 10); sensitive internal error traces omitted from production responses.

5.3 System Analysis and Feasibility

The project was technically feasible because React, Node.js, and MongoDB all have mature ecosystems and strong community support. It was also operationally feasible because the admin interface follows the existing showroom fulfillment process instead of forcing staff to learn a completely different workflow. Financially, the project is reasonable because the required cloud hosting cost is low compared with the value of automating product, order, and payment management.

5.4 System Design and Data Modeling

Figure 5.2 shows the entity relationship model for the database collections.

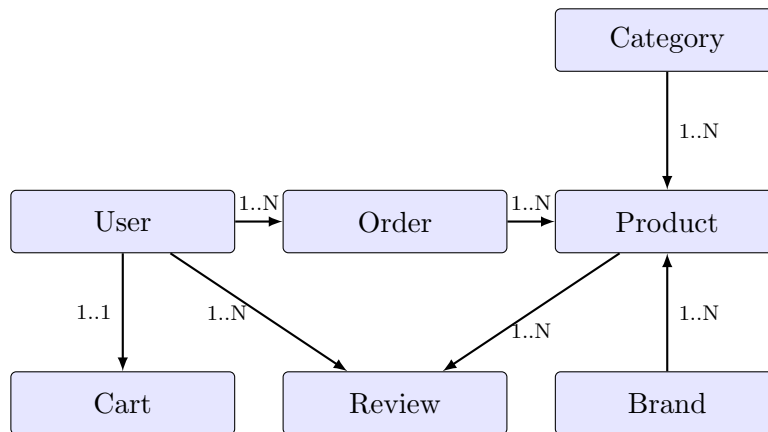


Figure 5.2: Entity Relationship Diagram (ERD)

5.4.1 MongoDB Collection Schema Summary

Table 5.1: MongoDB Collection Schema Summary

Collection	Key Fields
users	name, email, password (bcrypt), role, phone, address {division, district, street}, avatar
products	title, slug, brand, category (ref), price, discountPrice, stock, sku, images[], shortSpecs[], technicalSpecs (Map), reviews[], isFeatured, badge
categories	name, slug, parent (self-ref), image, isActive
orders	user (ref), orderItems[], shippingAddress, paymentMethod, paymentResult, itemsPrice, shippingPrice, totalPrice, isPaid, orderStatus, trackingNumber, statusHistory[]
coupons	code, discountType, discountValue, minOrderValue, maxUses, validUntil, isActive
carts	user (ref), items[] {product, qty, price}

5.5 UI Page Mockups

The following figures show screenshots from the completed application. I added a short description under each screenshot to explain the purpose of the screen and the main visible features.

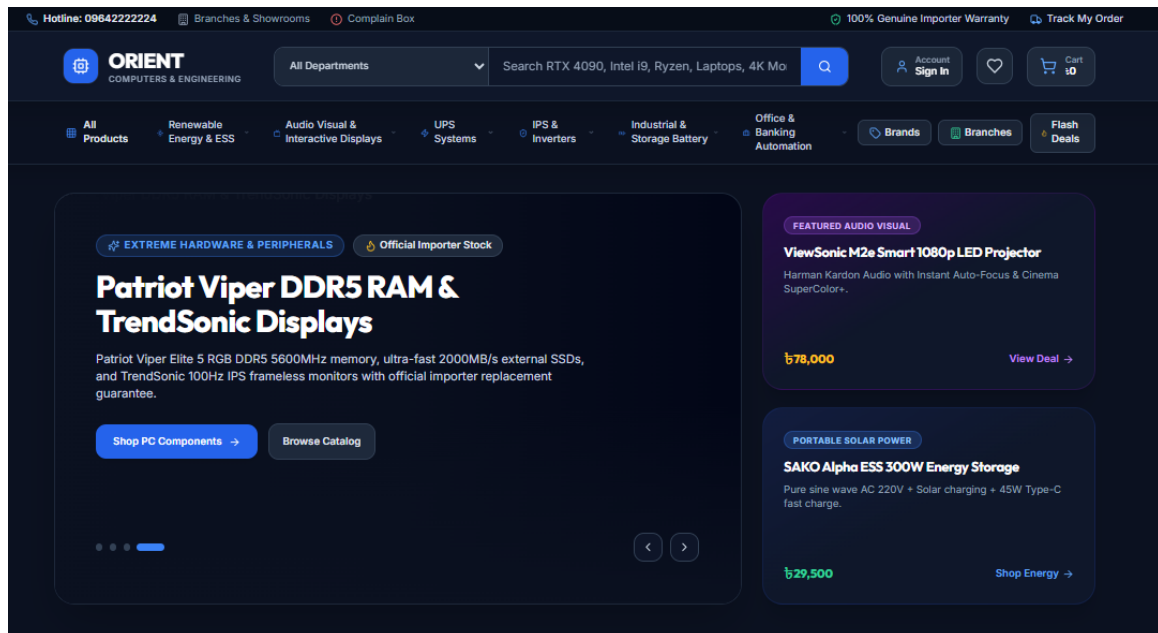


Figure 5.3: Homepage Screenshot

Description: The homepage screenshot shows the public entry point of the Orient Computer e-commerce platform. It highlights the navigation bar, search access, promotional banner, featured product sections, and product cards that guide customers toward browsing and purchasing computer, power backup, and accessory products.

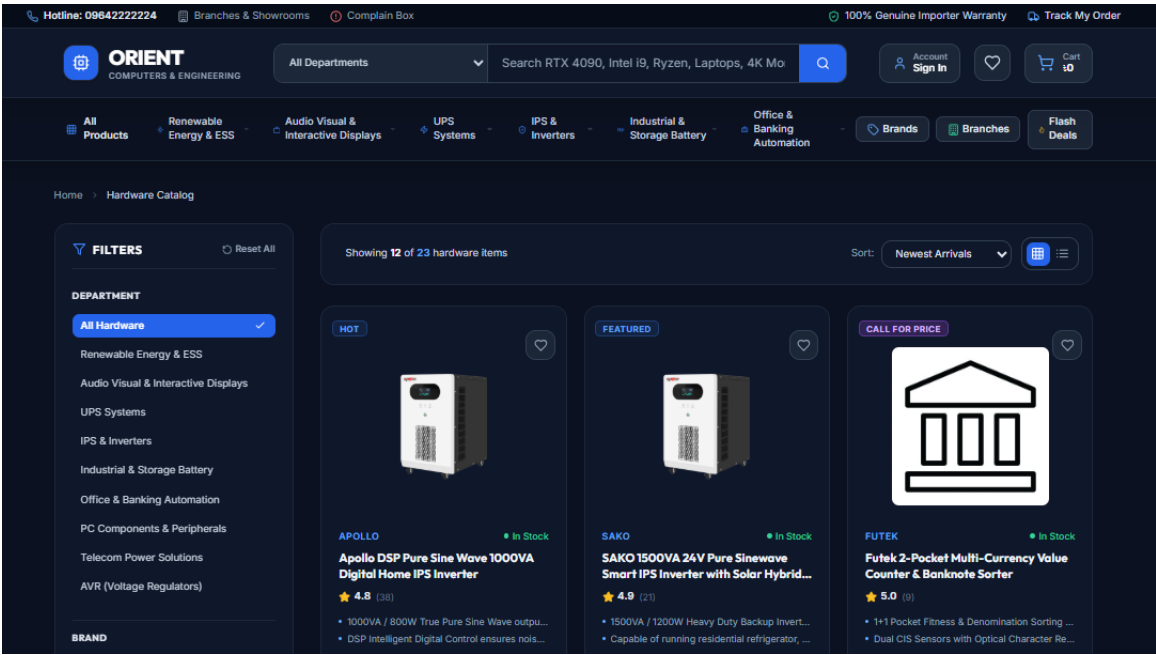


Figure 5.4: Shop Catalog Screenshot

Description: The shop catalog screenshot demonstrates the product browsing experience. It presents category and product discovery features, including filtering, sorting, product listings, prices, stock indicators, and add-to-cart actions that help customers compare items before purchase.

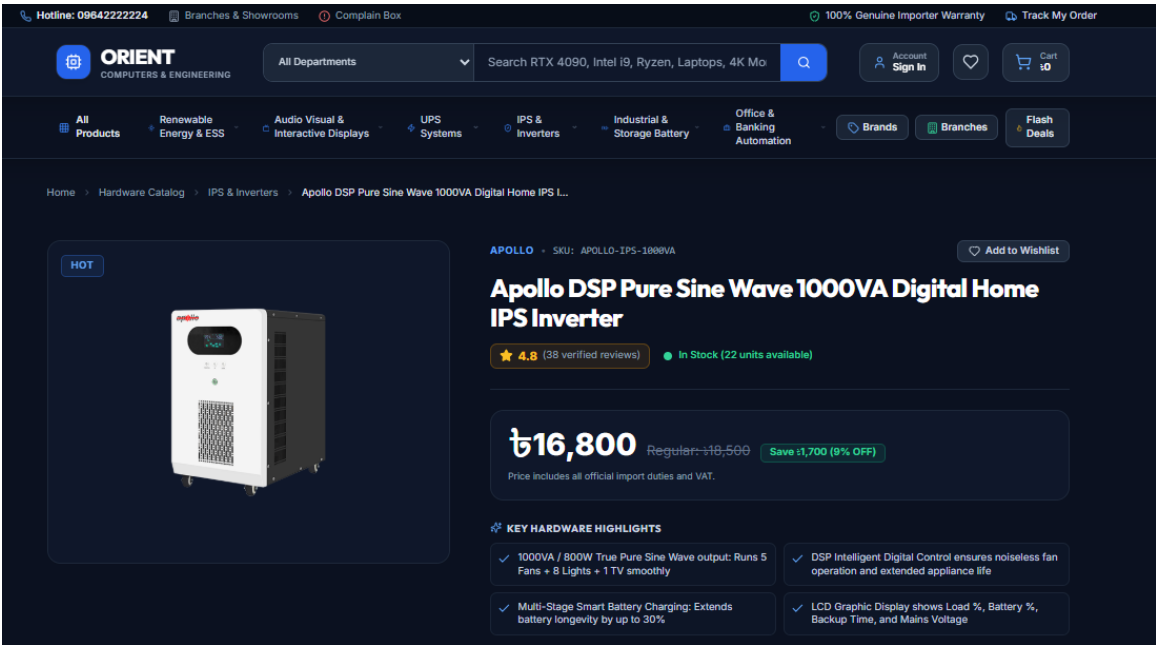


Figure 5.5: Product Detail Screenshot

Description: The product detail screenshot displays the dedicated product information page. It shows the selected product image, pricing, availability, technical details, quantity selection, and purchase actions, allowing customers to evaluate a product before adding it to the cart or proceeding directly to checkout.

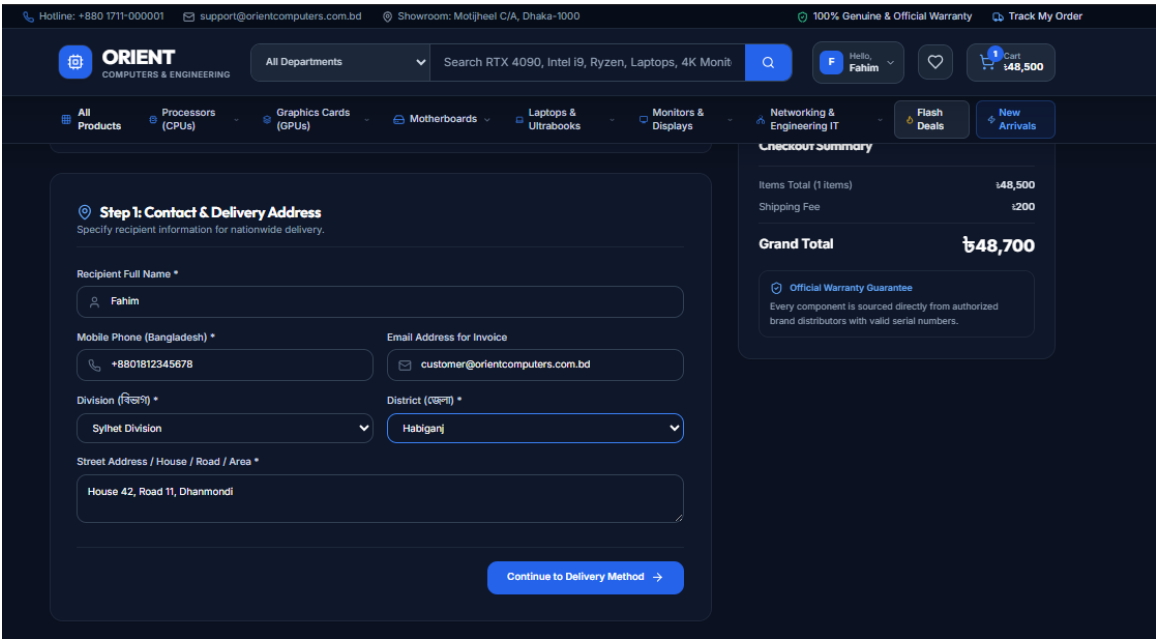


Figure 5.6: Checkout Screenshot

Description: The checkout screenshot illustrates the order completion workflow. It includes customer shipping information, order summary, payment method selection, coupon handling, and final order placement, supporting both cash-on-delivery and manual mobile financial service payment flows.

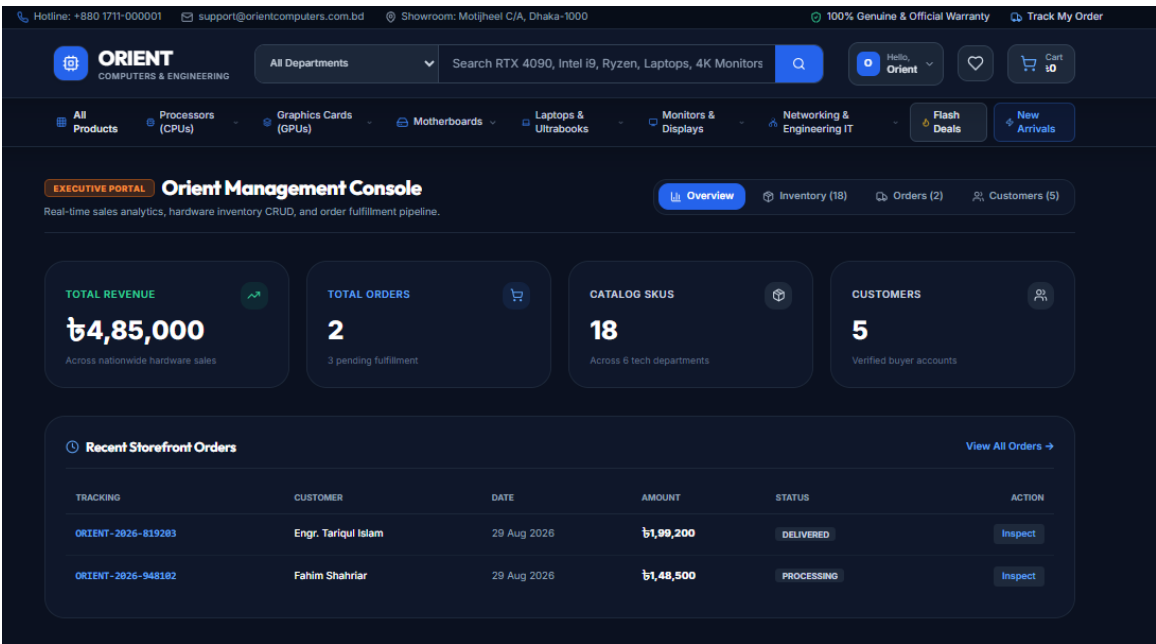


Figure 5.7: Admin Dashboard Screenshot

Description: The admin dashboard screenshot shows the management overview for store administrators. It summarizes key operational metrics such as revenue, orders, users, and products, while also giving quick access to charts and recent activity for monitoring business performance.

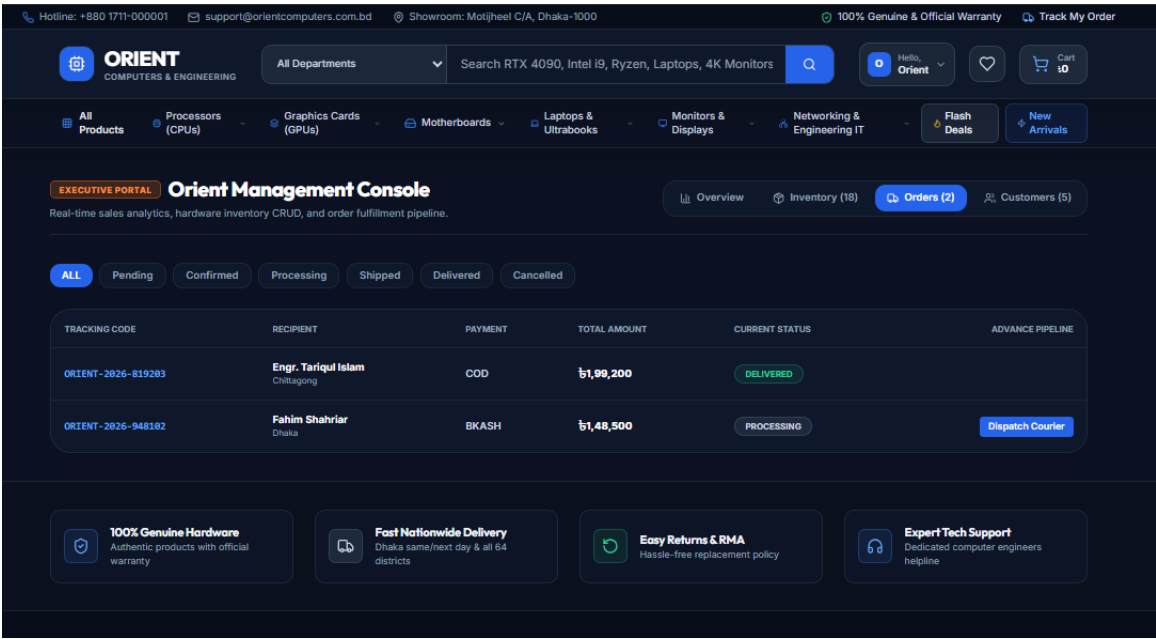


Figure 5.8: Admin Orders Screenshot

Description: The admin orders screenshot presents the order management interface. Administrators can review order records, inspect customer and payment details, track fulfillment status, and verify manually submitted payment screenshots before approving or updating an order.

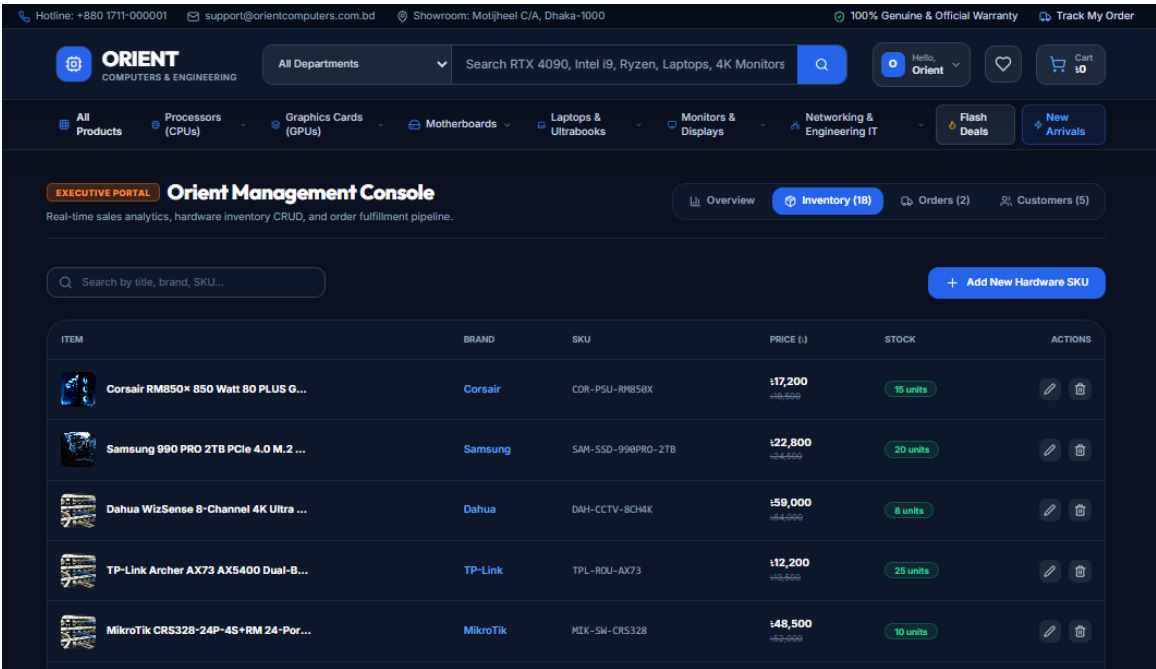


Figure 5.9: Admin Inventory Screenshot

Description: The admin inventory screenshot shows the product and stock management area. It helps administrators monitor available products, identify low-stock items, update product information, and keep the catalog aligned with current store inventory.

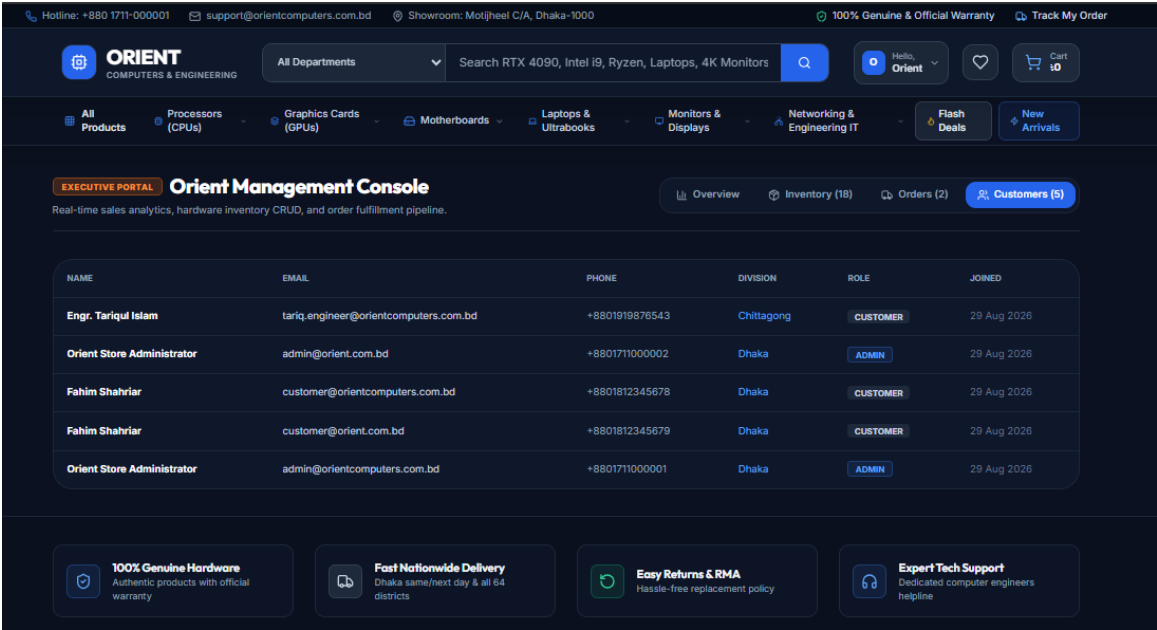


Figure 5.10: Admin Customers Screenshot

Description: The admin customers screenshot displays the customer management view. It provides administrators with an overview of registered users, customer contact information, account roles, and related activity useful for support and order follow-up.

5.6 Order State Machine

Figure 5.11 shows the finite-state machine used for order status changes on the server.

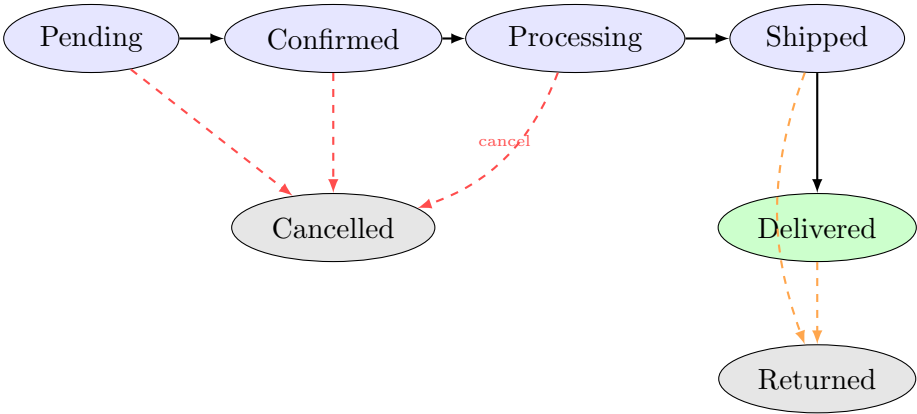


Figure 5.11: Order Lifecycle Finite-State Machine

Table 5.2: Permitted Order Status State Transitions

Current Status	Permitted Subsequent Statuses
Pending	Confirmed, Cancelled
Confirmed	Processing, Cancelled
Processing	Shipped, Cancelled
Shipped	Delivered, Returned
Delivered	Returned
Cancelled / Returned	(Terminal States – None)

5.7 Key Code Implementations

5.7.1 User Schema with bcrypt Password Hashing

The User Mongoose schema enforces server-side password hashing using bcrypt before saving to the database, preventing plain-text credential exposure. Listing 5.1 shows the complete model.

```

1 import mongoose from 'mongoose';
2 import bcrypt from 'bcryptjs';
3
4 const userSchema = new mongoose.Schema({
5   name: { type: String, required: true, trim: true, maxlength: 50 },
6   email: {
7     type: String, required: true, unique: true, lowercase: true,
8     match: [/^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
9       'Please provide a valid email'],
10  },
11  password: {
12    type: String, required: true, minlength: 6,
13    select: false, // excluded from queries by default
14  },
15  role: { type: String, enum: ['customer','admin'], default: 'customer' },
16  phone: { type: String, default: '' },
17  avatar: { type: String, default: '' },
18  address: {
19    division: { type: String, default: 'Dhaka' },
20    district: { type: String, default: 'Dhaka' },
21    street: { type: String, default: '' },
22    postalCode: { type: String, default: '' },
23    country: { type: String, default: 'Bangladesh' },
24  },
25 }, { timestamps: true });
26
27 // Hash password before saving if modified
28 userSchema.pre('save', async function (next) {
29   if (!this.isModified('password')) return next();
30   const salt = await bcrypt.genSalt(10);
31   this.password = await bcrypt.hash(this.password, salt);
32   next();
33 });
34
35 // Instance method for login password comparison

```

```

36 userSchema.methods.matchPassword = async function (entered) {
37   return await bcrypt.compare(entered, this.password);
38 };
39
40 export default mongoose.model('User', userSchema);

```

Listing 5.1: User.js – Mongoose Schema with bcrypt Pre-Save Hook

5.7.2 JWT Authentication Middleware

Listing 5.2 presents the three-tier middleware chain: mandatory `protect`, guest-tolerant `optionalProtect`, and admin privilege guard.

```

1 import jwt from 'jsonwebtoken';
2 import User from '../models/User.js';
3
4 // Mandatory JWT authentication
5 export const protect = async (req, res, next) => {
6   let token;
7   if (req.headers.authorization?.startsWith('Bearer ')) {
8     try {
9       token = req.headers.authorization.split(' ')[1];
10      const decoded = jwt.verify(token,
11        process.env.JWT_SECRET || 'orient_jwt_secret_key_2026'
12      );
13      const user = await User.findById(decoded.id).select('-password'
14    );
15      if (!user) {
16        res.status(401);
17        return next(new Error('User not found. Authorization failed.'
18      ));
19    }
20    req.user = user;
21    return next();
22    } catch {
23      res.status(401);
24      return next(new Error('Invalid or expired token. Not authorized
25    '));
26    }
27  }
28  res.status(401);
29  return next(new Error('No token provided. Not authorized.'));
30 };
31
32 // Optional: populates req.user if token present, allows guests
33 // otherwise
34 export const optionalProtect = async (req, res, next) => {
35   if (req.headers.authorization?.startsWith('Bearer ')) {
36     try {
37       const token = req.headers.authorization.split(' ')[1];
38       const decoded = jwt.verify(token,
39         process.env.JWT_SECRET || 'orient_jwt_secret_key_2026'
40       );
41       const user = await User.findById(decoded.id).select('-password'
42     );
43     if (user) req.user = user;
44   } catch { req.user = null; }
45 }

```

```

41   return next();
42 };
43
44 // Administrator privilege guard
45 export const admin = (req, res, next) => {
46   if (req.user?.role === 'admin') return next();
47   res.status(403);
48   return next(new Error('Access denied. Administrator privileges
49   required.'));

```

Listing 5.2: authMiddleware.js – JWT Protection and Admin Guard

5.7.3 Product Schema with Virtuals and Text Index

Listing 5.3 demonstrates the Product schema with a Map-typed `technicalSpecs` field for flexible hardware specifications, computed virtuals for stock status and discount percentage, and a compound text index for full-text catalog search.

```

1  const productSchema = new mongoose.Schema({
2    title:      { type: String, required: true, trim: true },
3    slug:      { type: String, required: true, unique: true,
4      lowercase: true },
5    brand:     { type: String, required: true },
6    category:  { type: mongoose.Schema.Types.ObjectId,
7      ref: 'Category', required: true },
8    price:     { type: Number, required: true, min: 0 },
9    discountPrice: { type: Number, default: 0 },
10   stock:     { type: Number, required: true, min: 0, default: 0
11   },
12   sku:       { type: String, required: true, unique: true,
13     uppercase: true },
14   images:    { type: [String], required: true },
15   shortSpecs: { type: [String], default: [] },
16   // Flexible key-value map for diverse hardware spec sheets
17   technicalSpecs: { type: Map, of: String, default: {} },
18   warranty:    { type: String, default: 'Official Brand Warranty'
19   },
20   reviews:    [reviewSchema],
21   isFeatured:  { type: Boolean, default: false },
22   badge: {
23     type: String,
24     enum: ['', 'HOT', 'SALE', 'NEW', 'GAMING', 'OFFER', 'FEATURED'],
25     default: '',
26   },
27 }, { timestamps: true, toJSON: { virtuals: true } });
28
29 // Virtual: inStock computed from stock quantity
30 productSchema.virtual('inStock').get(function () {
31   return this.stock > 0;
32 });
33
34 // Virtual: discount percentage calculation
35 productSchema.virtual('discountPercentage').get(function () {
36   if (this.discountPrice && this.discountPrice < this.price)
37     return Math.round(((this.price - this.discountPrice) / this.price
38     ) * 100);

```

```

34   return 0;
35 });
36
37 // Compound text index for full-text catalog search
38 productSchema.index({
39   title: 'text', brand: 'text',
40   shortSpecs: 'text', sku: 'text',
41 });

```

Listing 5.3: Product.js – Schema with Virtuals and Full-Text Index

5.7.4 Order Creation with Guest Checkout and Atomic Stock Decrement

Listing 5.4 shows the order creation controller. It supports both authenticated users and guests (by provisioning implicit accounts), performs atomic stock decrements per purchased SKU, and returns an auto-generated tracking number of the form ORIENT-YYYY-XXXXXX.

```

1  export const createOrder = async (req, res, next) => {
2    try {
3      const { orderItems, shippingAddress, paymentMethod,
4              itemsPrice, shippingPrice, discountPrice, totalPrice } =
5        req.body;
6
7      if (!orderItems?.length) {
8        res.status(400);
9        return next(new Error('No order items provided.'));
10     }
11
12     let orderUser = req.user;
13     let autoCreatedToken = null;
14
15     // Guest checkout: find existing user by phone/email or create
16     // one
17     if (!orderUser) {
18       const cleanPhone = (shippingAddress.phone || '').replace
19         (/[^0-9+]/g, '');
20       const guestEmail = (shippingAddress.email ||
21         'guest_${cleanPhone || Date.now()}@orientcomputers.com.bd').
22         toLowerCase();
23
24       let user = await User.findOne({
25         $or: [{ email: guestEmail },
26               ...(cleanPhone ? [{ phone: cleanPhone }] : [])],
27       });
28
29       if (!user) {
30         const randomSecret = Math.random().toString(36).slice(-8);
31         user = await User.create({
32           name: shippingAddress.fullName,
33           email: guestEmail,
34           password: 'Orient#${randomSecret}2026',
35           phone: cleanPhone || '',
36           role: 'customer',
37         });
38       }
39     }
40   }

```

```

35     orderUser = user;
36     autoCreatedToken = generateToken(user._id);
37   }
38
39   const order = new Order({
40     user: orderUser._id, orderItems, shippingAddress,
41     paymentMethod, itemsPrice, shippingPrice, discountPrice,
42     totalPrice,
43     isPaid: paymentMethod !== 'COD',
44     paidAt: paymentMethod !== 'COD' ? new Date() : null,
45     orderStatus: 'Pending',
46   });
47
48   const createdOrder = await order.save();
49
50   // Atomic stock decrement for each purchased SKU
51   for (const item of orderItems) {
52     if (item.product) {
53       await Product.findByIdAndUpdate(item.product, {
54         $inc: { stock: -Number(item.qty || 1) },
55       });
56     }
57   }
58
59   const payload = { success: true, order: createdOrder };
60   if (autoCreatedToken) {
61     payload.token = autoCreatedToken;
62     payload.user = { _id: orderUser._id, name: orderUser.name,
63                     email: orderUser.email, role: orderUser.role
64   };
65   }
66   res.status(201).json(payload);
67 } catch (error) { next(error); }
68 };

```

Listing 5.4: orderController.js – createOrder with Guest Account Provisioning and Stock Decrement

5.7.5 Admin Analytics Aggregation Pipeline

Listing 5.5 presents the dashboard analytics controller using MongoDB’s aggregation framework with parallel Promise.all execution for optimal performance.

```

1 export const getAdminAnalytics = async (req, res, next) => {
2   try {
3     // Run all count queries in parallel for performance
4     const [totalOrders, pendingOrders, deliveredOrders,
5           totalProducts, lowStockProducts, totalCustomers] =
6       await Promise.all([
7         Order.countDocuments({}),
8         Order.countDocuments({ orderStatus: 'Pending' }),
9         Order.countDocuments({ orderStatus: 'Delivered' }),
10        Product.countDocuments({}),
11        Product.countDocuments({ stock: { $lte: 5 } }),
12        User.countDocuments({ role: 'customer' }),
13      ]);
14

```



```
15 // Aggregation pipeline: revenue excluding cancelled orders
16 const revenueAgg = await Order.aggregate([
17   { $match: { orderStatus: { $ne: 'Cancelled' } } },
18   { $group: { _id: null, totalRevenue: { $sum: '$totalPrice' } } }
19 ],
20 );
21 const totalRevenue = revenueAgg[0]?.totalRevenue || 0;
22
23 const recentOrders = await Order.find({})
24   .populate('user', 'name email')
25   .sort({ createdAt: -1 })
26   .limit(6);
27
28 res.status(200).json({
29   success: true,
30   analytics: {
31     totalRevenue, totalOrders, pendingOrders,
32     deliveredOrders, totalProducts, lowStockProducts,
33     totalCustomers, recentOrders,
34   },
35 });
36 } catch (error) { next(error); }
37 };
```

Listing 5.5: orderController.js – Admin Dashboard Analytics with Aggregation Pipeline

5.8 Implementation and Testing

Testing covered the most important parts of the system: price and coupon calculations, API endpoints connected to MongoDB Atlas, and security checks for token manipulation and unauthorized admin access.

Table 5.3: Key Test Case Summary

Test ID	Test Scenario	Result
TC-01	POST /api/auth/register with valid payload creates user and returns JWT	PASS
TC-02	POST /api/auth/login with wrong password returns 401 Unauthorized	PASS
TC-03	GET /api/products with category filter returns only matching products	PASS
TC-04	POST /api/orders decrements stock atomically after successful creation	PASS
TC-05	PATCH /api/admin/orders/:id/status without admin role returns 403	PASS
TC-06	Manipulated totalPrice in request body is ignored; server recalculates	PASS
TC-07	Checkout with out-of-stock product returns 400 with descriptive error	PASS
TC-08	Guest checkout creates implicit customer account and returns JWT	PASS
TC-09	Invalid JWT token on protected route returns 401 Not Authorized	PASS
TC-10	Order tracking by valid tracking number returns masked order details	PASS

Chapter 6

Results and Analysis

The Orient Computer platform was built and tested with realistic product data across 8 parent categories. I also checked the main pages on different screen sizes to make sure the customer and admin interfaces were usable.

Table 6.1: System Performance and Latency Metrics

Transaction / Operation	Data Payload	Avg. Latency (ms)
Product Listing Query (filters & pagination)	~ 24 KB	380
Product Detail & Specification Retrieval	~ 8 KB	220
Order Creation (Validation, Price Freeze, DB Write)	~ 4 KB	740
Admin Aggregate Dashboard KPI Calculation	~ 12 KB	450

The benchmark results show that the main API responses stayed below the target limit of 3 seconds under realistic network conditions.

6.1 System Features Delivered

All five primary objectives defined in Chapter 1 were delivered:

- **Customer Storefront:** Responsive React 18 SPA with 14 pages, dynamic catalog, and multi-step checkout.
- **Localized Payments:** COD, bKash, and Nagad workflows with payment proof submission and admin verification queue.
- **Admin Dashboard:** Operational KPI cards, order management table, low-stock alerts, and product/category CRUD.
- **Security:** bcrypt password hashing, JWT Bearer scheme, role-based guards, Helmet.js HTTP headers.
- **Guest Checkout:** Implicit account creation that allows checkout without prior registration.

6.2 Deployment Architecture

Figure 6.1 illustrates the production deployment topology across cloud platforms.

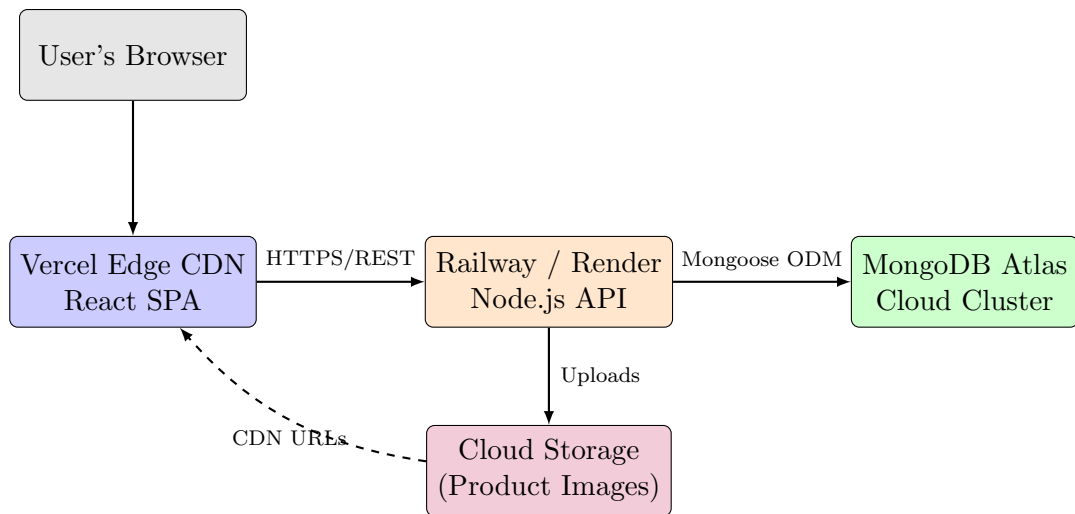


Figure 6.1: Production Deployment Architecture

Chapter 7

Project as Engineering Problem Analysis

7.1 Sustainability of the Project

The application codebase was organized with separation of concerns in mind. Database schemas, HTTP route controllers, middleware, and business logic were kept in separate files so the project would be easier to understand and extend later. The use of cloud hosting and MongoDB Atlas also reduces the need for maintaining dedicated physical infrastructure for a small business application.

7.2 Social and Environmental Considerations

By giving Orient Computer a digital storefront, customers from different districts in Bangladesh can check products and place orders without always visiting the showroom. This can reduce unnecessary travel for customers who only need product information or want to confirm availability before buying. The admin dashboard can also help staff become more comfortable with digital order and inventory management.

7.3 Ethical and Legal Considerations

Customer data is handled carefully in the system. Passwords are never stored in plain text, and payment screenshots submitted for MFS verification are available only to authorized administrators. The platform avoids collecting unnecessary analytics data beyond what is needed for normal operation.

Chapter 8

Lessons Learned

8.1 Problems Faced

- **MongoDB Transactions on Standalone Instances:** Multi-document transactions require replica set configurations. During local testing, this caused issues, so I added fallback rollback logic for cases where transactions were not available.
- **Cart Synchronization Race Conditions:** Merging a guest cart from the browser with an existing server-side cart was more complicated than expected. I had to compare items one by one so product quantities and stock limits stayed correct.
- **Windows Path Separators in Multer:** File uploads produced backslash path separators on Windows. These had to be normalized into forward slashes so uploaded image URLs worked consistently.
- **District-wise Delivery Fee Complexity:** Bangladesh's 64 districts required a structured delivery fee setup. I first considered hard-coded values, but later moved the fee matrix into the database so admins could update it without changing code.
- **JWT Storage Security Trade-offs:** Storing JWTs in `localStorage` is simple, but it can expose tokens if an XSS attack occurs. For the current version, this risk is reduced through security headers and input sanitization. A future version should move authentication to `HttpOnly` cookies.

8.2 Solutions and Retrospective Insights

One lesson I learned is that shared business rules should be kept in one place. Moving price and coupon calculations into centralized service modules reduced duplicate code and made testing easier. I also learned that writing the SRS before implementation was useful because it clarified the expected features and reduced confusion during development. In future versions, tools such as Redis or TanStack React Query could improve caching and client-side data fetching.

Chapter 9

Future Work and Conclusion

9.1 Future Work

Planned enhancements include:

1. Integrate automated payment gateways such as SSLCommerz or ShurjoPay to reduce the need for manual MFS screenshot verification.
2. Add SMS and email notifications when order status changes.
3. Build an interactive PC Builder tool with compatibility checking for DIY PC customers.
4. Move authentication from localStorage JWT storage to CSRF-protected `HttpOnly` cookies for better protection against token theft.
5. Add product recommendations based on order history and browsing behavior.
6. Add Progressive Web App (PWA) support for offline browsing and home screen installation.

9.2 Conclusion

The Orient Computer e-commerce platform shows how a local retail business can move important parts of its sales and order process online. The project combines a customer storefront, localized payment options, and an admin dashboard for managing products, orders, stock, and manual payment verification. Through this work, I was able to apply my academic knowledge to a real business problem and complete the main objectives of the internship. The MERN stack was a practical choice for the timeline because it allowed me to work across the frontend and backend using one main programming language.

Bibliography

- [1] a2i (Aspire to Innovate), Government of Bangladesh, “Aspire to Innovate Official Website,” accessed 2026. Available: <https://a2i.gov.bd/>
- [2] Orient Computers & Engineering, “Orient Computers & Engineering Official Website,” accessed 2026. Available: <https://orientcomputers.com.bd/>
- [3] Bangladesh Bank, “Payment and Settlement Systems,” Bangladesh Bank, accessed 2026. Available: <https://www.bb.org.bd/en/index.php/financialactivity/paysystems>
- [4] Star Tech Ltd., “Star Tech Online Shopping App,” Google Play Store listing, accessed 2026. Available: <https://play.google.com/store/apps/details?id=com.startech.shop>
- [5] Ryans Computers Ltd., “Online Payment Method,” Ryans Computers, accessed 2026. Available: <https://www.ryans.com/page/payment-method>
- [6] MongoDB Inc., “How To Use MERN Stack: A Complete Guide,” MongoDB Developer Resources, accessed 2026. Available: <https://www.mongodb.com/resources/languages/mern-stack-tutorial>
- [7] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015. Available: <https://datatracker.ietf.org/doc/html/rfc7519>
- [8] OWASP Foundation, “OWASP Top Ten,” accessed 2026. Available: <https://owasp.org/Top10/>
- [9] OpenJS Foundation, “Routing: Express.js 5.x,” accessed 2026. Available: <https://expressjs.com/en/5x/guide/routing/>
- [10] Meta Platforms, “React Documentation,” accessed 2026. Available: <https://react.dev/>
- [11] Automattic, “Mongoose ODM Documentation,” accessed 2026. Available: <https://mongoosejs.com/docs/>
- [12] N. Provos and D. Mazieres, “A Future-Adaptable Password Scheme,” in *Proceedings of the USENIX Annual Technical Conference*, 1999. Available: <https://www.usenix.org/conference/1999-usenix-annual-technical-conference/future-adaptable-password-scheme>